

Week 8 Tutorial

Introduction

This workshop will help you familiarize yourself with: non-stationary time-series; the Augmented Dickey Fuller test and; forecasting autoregressive models in R.

NOTE: We have used `echo = FALSE` in some code chunks so that they do not all render in the handout, but they are included in the .qmd file. This is a similar approach you should use in the assignment - you do not need to provide any R code.

We also use the [here](#) package so that we can call the `Correlogram` function globally. If you are not using a file structure in your Project, you do not need this. If you need help with this, ask in a consultation - do not worry about it today.

Question 1

Consider an AR(1) process given by

$$y_t = \phi_1 y_{t-1} + \varepsilon_t,$$

where $\varepsilon_t \sim_{iid} WN(0, \sigma_\varepsilon^2)$ (white noise).

Recall from the Week 7 tutorial, in order for this process to be stationary we require that condition $|\phi_1| < 1$ be met. Even when $|\phi_1| < 1$, the absolute value of ϕ_1 determines the ‘persistence’ of the process.

We will use the [Week8.xlsx](#) file from Moodle. The variable **y1t** is an AR(1) process with $\phi = 0.8$. We have 499 observations (we lose one to create the lag variable).

Read in the data:

```
library(readxl)
library(xts)

data.df <- read_excel("Week8.xlsx")

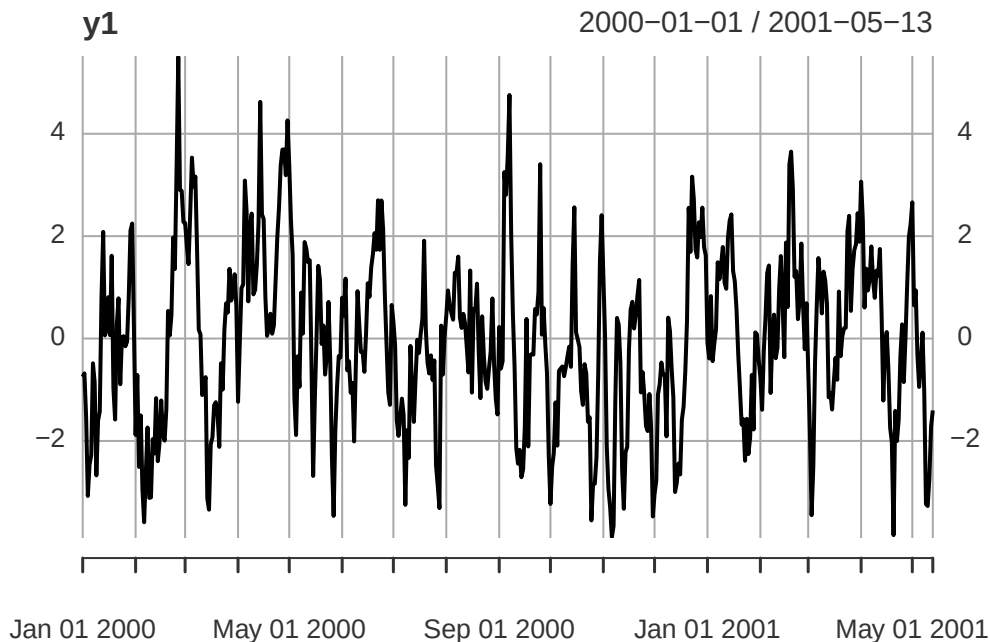
# create a random time index first because xts needs an index.
```

```

random_date <- as.Date("2000-01-01") + 0:(nrow(data.df) - 1)
data.xts <- xts(data.df, order.by = random_date)

y1 <- data.xts$Y1T
plot(y1)

```



Use data up to point 449 to train/fit the model, and then predict **y1t** up to the end of the sample (*i.e.* estimation sample: 1 449; forecast sample: 450-499). Remember, we have the observed values for the last 50 observations, so we will be able to compare our forecasts we generate to the actual values.

```

H <- 50
# y1 <- y1[-1] # remove the first observation as it contains zero.
n <- NROW(y1)

train_xts <- y1[1:(n - H)]
test_xts  <- y1[(n - H + 1):n]

```

Now estimate the AR(1) model over the training data set. To do this in R, we use the `arima` function.

```

#=====
# AR(1)
#=====
fit_ar1 <- arima(as.numeric(train_xts), order = c(1, 0, 0),

```

```
include.mean = TRUE)
fit_ar1
```

Call:

```
arima(x = as.numeric(train_xts), order = c(1, 0, 0), include.mean = TRUE)
```

Coefficients:

```
      ar1  intercept
      0.7848    -0.0426
s.e.  0.0291     0.2270
```

```
sigma^2 estimated as 1.088:  log likelihood = -656.55,  aic = 1319.1
```

The estimated AR(1) model in R is written as mean form

$$y_t - \mu = \phi_1 (y_{t-1} - \mu) + \varepsilon_t$$

and so

$$y_t + 0.0426 = 0.7848 (y_{t-1} + 0.0426) + \hat{\varepsilon}_t$$

This is just a de-meanned version of the AR(1) specification we did last week.

Now we want to use this estimated model to forecast over the test data.

```
pred_ar1 <- predict(fit_ar1, n.ahead = H)

fc_ar1 <- xts(pred_ar1$pred, order.by = index(test_xts))
colnames(fc_ar1) <- "Forecast"
```

Since we have the actual observations, we can calculate some error statistics for our forecasts.

```
compare_ar1 <- merge(test_xts, fc_ar1)
colnames(compare_ar1) <- c("Actual", "Forecast")
head(compare_ar1)
```

	Actual	Forecast
2001-03-25	2.3970626	1.6317847
2001-03-26	0.5398486	1.2715203
2001-03-27	1.3535944	0.9887723
2001-03-28	1.7259254	0.7668618
2001-03-29	1.8417429	0.5926987
2001-03-30	2.4494056	0.4560094

```

err_ar1 <- compare_ar1$Actual - compare_ar1$Forecast
abs_err_ar1 <- abs(err_ar1)
sq_err_ar1 <- err_ar1^2

MAE_ar1 <- mean(as.numeric(abs_err_ar1), na.rm = TRUE)
MSE_ar1 <- mean(as.numeric(sq_err_ar1), na.rm = TRUE)
RMSE_ar1 <- sqrt(MSE_ar1)

results_ar1 <- data.frame(
  MAE_ar1 = MAE_ar1,
  MSE_ar1 = MSE_ar1,
  RMSE_ar1 = RMSE_ar1
)

results_ar1

```

```

      MAE_ar1 MSE_ar1 RMSE_ar1
1 1.305792 2.476279 1.57362

```

These error statistics do not really tell us anything until we have another model to compare them to.

We can plot the Actual vs Forecasts, with a 95% CI. The code for the plot is (the Monash University enterprise version of Gemini (<https://gemini.google.com/app>) was used to generate the code for the plot):

```

library(ggplot2)
library(dplyr)
library(zoo)

# 1. Create a data frame for the Test period only
# We combine the actual test values and the forecast results
plot_test_data <- data.frame(
  Index = index(test_xts),
  Actual = as.numeric(test_xts),
  Forecast = as.numeric(pred_ar1$pred),
  Lower = as.numeric(pred_ar1$pred - 1.96 * pred_ar1$se),
  Upper = as.numeric(pred_ar1$pred + 1.96 * pred_ar1$se)
)

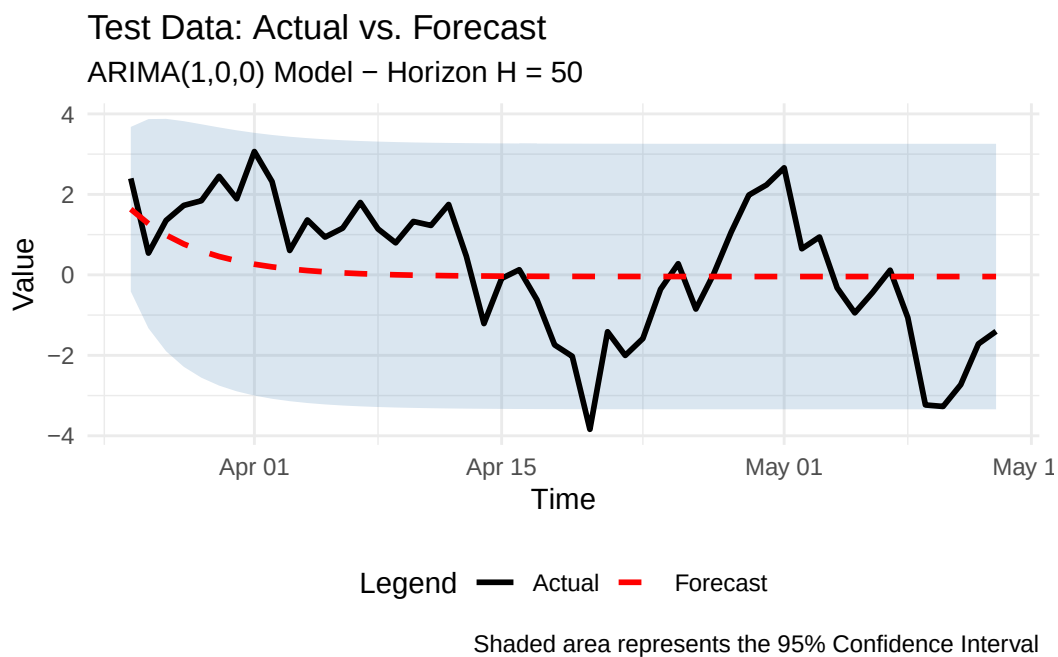
# 2. Generate the plot
ggplot(plot_test_data, aes(x = Index)) +
  # Add the 95% Confidence Interval ribbon
  geom_ribbon(aes(ymin = Lower, ymax = Upper), fill = "steelblue",

```

```

    alpha = 0.2) +
# Add the Actual values line
geom_line(aes(y = Actual, color = "Actual"), linewidth = 1) +
# Add the Forecasted values line
geom_line(aes(y = Forecast, color = "Forecast"), linewidth = 1,
          linetype = "dashed") +
# Refine scales and labels
scale_color_manual(values = c("Actual" = "black", "Forecast" = "red")) +
labs(
  title = "Test Data: Actual vs. Forecast",
  subtitle = paste("ARIMA(1,0,0) Model - Horizon H =", H),
  x = "Time",
  y = "Value",
  color = "Legend",
  caption = "Shaded area represents the 95% Confidence Interval"
) +
theme_minimal() +
theme(legend.position = "bottom")

```



We can see from the forecast that there is a bit of movement in the beginning of the forecast, but it levels off to the mean the further we get away from the training data. The 95% confidence interval starts out narrow and becomes quite wide the further out we forecast. These results are not surprising (refer to the derivations at the end of the document).

The confidence interval captures most of the actual observations, except for a low dip just after April 15.

Now we will repeat the above by estimating an AR(2) model (we could of course have done everything all at once, but it is better to walk through one example in the first instance). We need to decide which model performs best, so we will compare the accuracy of the forecasts (use RMSE, MAE and MAPE). Is the result surprising?

The estimated AR(2) output is:

```
#=====
# AR(2)
#=====
fit_ar2 <- arima(as.numeric(train_xts), order = c(2, 0, 0), include.mean = TRUE)
fit_ar2
```

Call:

```
arima(x = as.numeric(train_xts), order = c(2, 0, 0), include.mean = TRUE)
```

Coefficients:

	ar1	ar2	intercept
	0.7986	-0.0175	-0.0433
s.e.	0.0473	0.0473	0.2231

sigma^2 estimated as 1.088: log likelihood = -656.48, aic = 1320.96

```
pred_ar2 <- predict(fit_ar2, n.ahead = H)

fc_ar2 <- xts(pred_ar2$pred, order.by = index(test_xts))
colnames(fc_ar2) <- "Forecast"

compare_ar2 <- merge(test_xts, fc_ar2)
colnames(compare_ar2) <- c("Actual", "Forecast")
# head(compare_ar2)

err_ar2 <- compare_ar2$Actual - compare_ar2$Forecast
abs_err_ar2 <- abs(err_ar2)
sq_err_ar2 <- err_ar2^2

MAE_ar2 <- mean(as.numeric(abs_err_ar2), na.rm = TRUE)
MSE_ar2 <- mean(as.numeric(sq_err_ar2), na.rm = TRUE)
RMSE_ar2 <- sqrt(MSE_ar2)
```

As with the AR(1) model above, the estimated AR(2) model in R is written as mean form

$$y_t - \mu = \phi_1 (y_{t-1} - \mu) + \phi_2 (y_{t-2} - \mu) + \varepsilon_t$$

and so

$$y_t + 0.0433 = 0.7986(y_{t-1} + 0.0433) - 0.0175(y_{t-2} + 0.0433) + \hat{\varepsilon}_t$$

Now that we have two models, we can compare the forecast errors to help select the model that predicts the test data the best.

```
#=====
# Compare
#=====
acc_compare <- rbind(
  AR1 = c(MAE = MAE_ar1, RMSE = RMSE_ar1),
  AR2 = c(MAE = MAE_ar2, RMSE = RMSE_ar2)
)

print(acc_compare)
```

	MAE	RMSE
AR1	1.305792	1.573620
AR2	1.309450	1.577056

When comparing fit statistics (specifically the MAE and RMSE in this case), the lower the number, the better the fit. The AR(1) forecast has (slightly) lower values for both of these statistics, implying that it has superior forecasting performance than the AR(2) model.

This is not surprising as the series was generated as an AR(1) model, so the AR(2) would be overfitting the time dynamics. It's forecasts would be using an extra, unnecessary term. Also, we lose an observation due to the extra lag term, so the estimates will be (slightly) less precise.

Question 2

When $\phi_1 \geq 1$, then y_t in the AR(1) model is said to be a non-stationary or explosive time series process. To illustrate this point, in the [Week8.xlsx](#) file, **y2t** has $\phi = 1$ (*i.e.* it has a unit root) and **y3t** has $\phi = 1.2$ (*i.e.* it is an explosive series).

1. Produce the correlograms for both of the time series. Briefly discuss.

```
# Use the here package to call the file globally

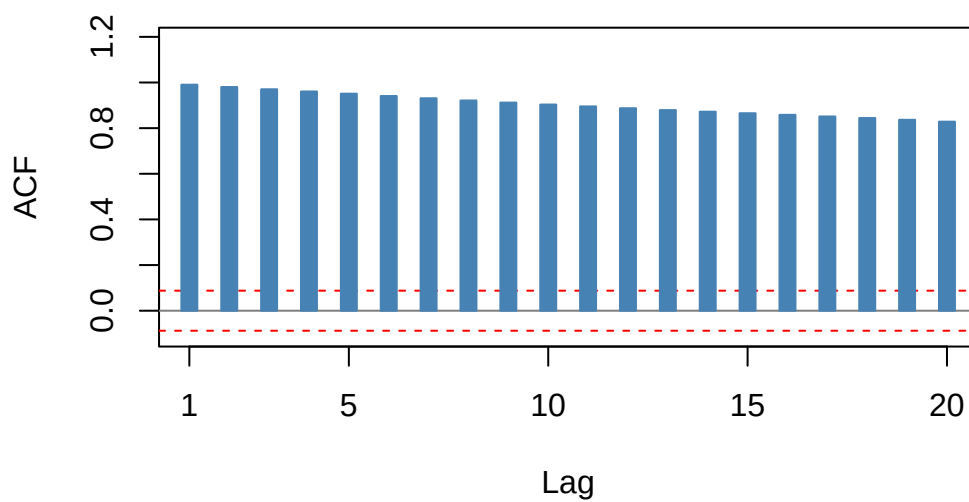
source(here::here("Correlogram.R"))

# Otherwise, comment out the line above and use this:
# source("Correlogram.R")

# Y2T
correlogram(data.xts$Y2T, max.lag = 20)
```

Lag	AC	Q	Prob
1	0.9904	492.4105	4.259e-109
2	0.9802	975.6816	1.360e-212
3	0.9703	1450.1854	3.795e-314
4	0.9607	1916.3100	0.000e+00
5	0.9511	2374.0880	0.000e+00
6	0.9408	2822.9118	0.000e+00
7	0.9310	3263.2902	0.000e+00
8	0.9212	3695.3553	0.000e+00
9	0.9119	4119.6629	0.000e+00
10	0.9035	4536.9547	0.000e+00
11	0.8953	4947.5601	0.000e+00
12	0.8873	5351.7101	0.000e+00
13	0.8791	5749.2659	0.000e+00
14	0.8721	6141.3453	0.000e+00
15	0.8653	6528.1343	0.000e+00
16	0.8583	6909.4710	0.000e+00
17	0.8511	7285.2183	0.000e+00
18	0.8443	7655.6874	0.000e+00
19	0.8366	8020.2404	0.000e+00
20	0.8284	8378.4375	0.000e+00

ACF (Correlogram)

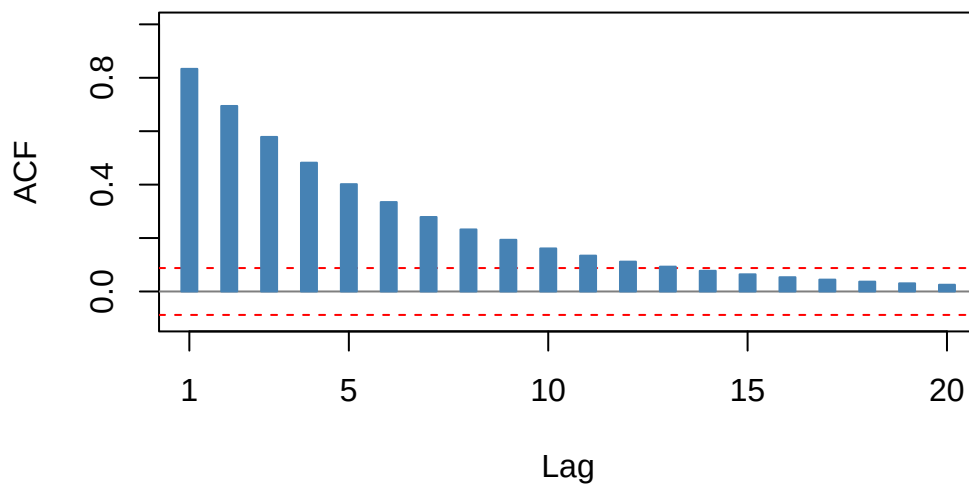


```
# Y3T
correlogram(data.xts$Y3T, max.lag = 20)
```

Lag	AC	Q	Prob
-----	----	---	------

1	0.8333	348.5775	8.648e-78
2	0.6944	591.0955	4.418e-129
3	0.5786	759.8151	2.244e-164
4	0.4821	877.1852	1.461e-188
5	0.4017	958.8267	4.923e-205
6	0.3346	1015.6090	3.762e-216
7	0.2788	1055.0957	1.496e-223
8	0.2322	1082.5497	2.248e-228
9	0.1934	1101.6331	2.061e-231
10	0.1611	1114.8940	3.248e-233
11	0.1341	1124.1052	3.650e-234
12	0.1116	1130.5004	1.588e-234
13	0.0929	1134.9377	1.750e-234
14	0.0773	1138.0140	3.642e-234
15	0.0642	1140.1448	1.167e-233
16	0.0534	1141.6189	5.000e-233
17	0.0443	1142.6370	2.598e-232
18	0.0367	1143.3389	1.531e-231
19	0.0304	1143.8216	9.761e-231
20	0.0252	1144.1525	6.526e-230

ACF (Correlogram)



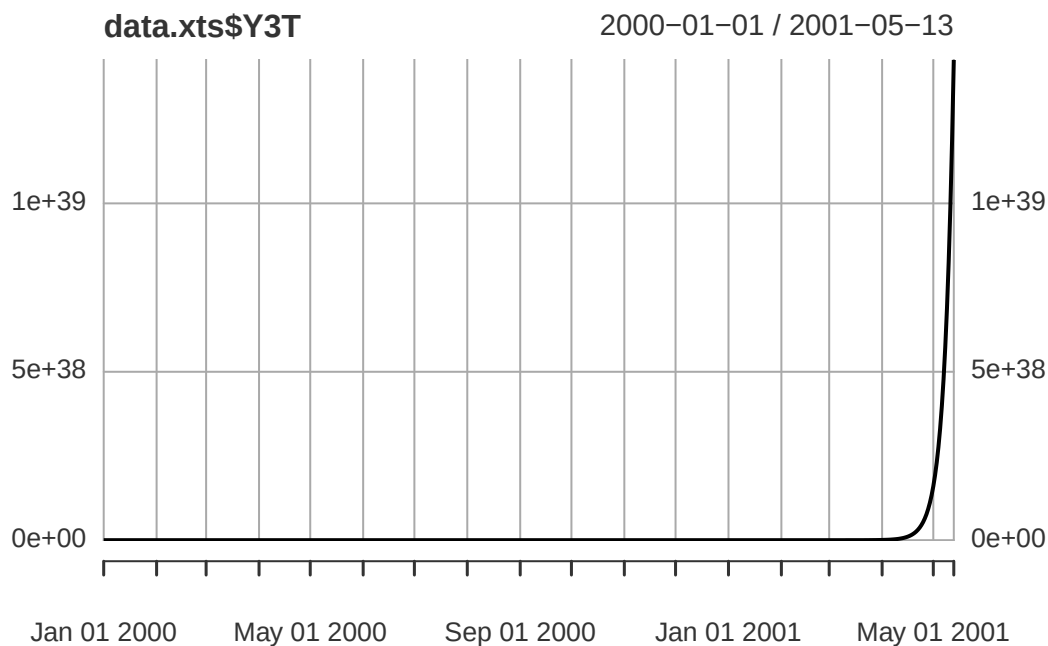
```
# Optional - put both correlograms in one plot

# par(mfrow = c(1,2)) # graphics to draw 1 row and 2 columns of plots
# correlogram(data.xts$Y2T, max.lag = 20, main.acf = "Correlogram of Y2T")
# correlogram(data.xts$Y3T, max.lag = 20, main.acf = "Correlogram of Y3T")
```

The correlogram for the random walk model ($\phi = 1$) shows that the correlations do not approach 0. There is extremely strong persistence in the series. For the explosive series, the correlogram looks like it approaches zero quickly and looks quite well-behaved.

In fact, **y3t** is not well-behaved. If we look at the line plot, the series explodes, so it has an infinite variance. We cannot analyse this data.

```
# plot of Y3T
par(mfrow = c(1,1))
plot(data.xts$Y3T)
```



You can see that the series increases dramatically (1e+39 is apparently called a duodecillion - and it goes higher than that, even after starting at 0).

2. In the case when $\phi_1 = 1.0$ (*i.e.* for **y3t**), conduct an Augmented Dickey Fuller test to formally establish whether the time series contains a unit root.

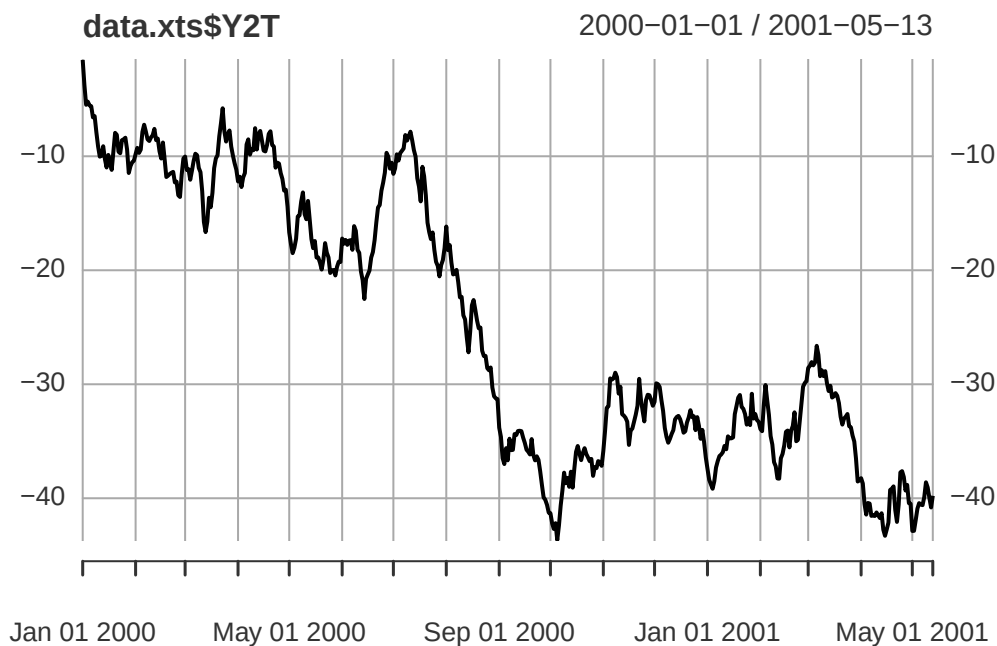
We know that we have a unit root in the series because it was created that way. In practice, we would suspect a unit root because of the pattern in the correlogram. Essentially what we want to do is to do a t-test to see if $\phi = 1$ or not, using the ADF. We will be using the R package **urca** to do this (so you need to install this package before you start), and it is important that you know the form of the ADF that the package uses, as this will determine the steps that you need to write down when performing the test. We have provided a short guide in the code, but you can also use the help function in R.

It is also important to remember that while R does give us a calculated test statistic, the critical values for this test statistic have been tabulated using simulations. So we will have

to compare the calculated test statistic to the ones provided in the output. We have no p-value for this test.

First you need to decide if you use an intercept, or an intercept with trend in the ADF test. If there is a trend in the series, use "trend" as an option. Otherwise use "drift" (we will never use "none").

```
# plot of Y2T
par(mfrow = c(1,1))
plot(data.xts$Y2T) # shows that Y2T contains a trend term
```



We need to look at the line plot of the **y2t** series to determine if we should use a trend term or not. There is a trend in the series, so we should use the trend option in the test.

```
library(urca)

#####
# R (`ur.df`) argument |
# -----
# `type = "none"`      |
# `type = "drift"`     |
# `type = "trend"`     |
# -----
# `selectlags = "AIC"` |
# `selectlags = "BIC"` |
# -----
```

```
# trend
# maximum lag = 17
adf_level_BIC <- ur.df(data.xts$Y2T, type = "trend", selectlags = "BIC", lags = 17)
summary(adf_level_BIC)
```

```
#####
# Augmented Dickey-Fuller Test Unit Root Test #
#####
```

Test regression trend

Call:

```
lm(formula = z.diff ~ z.lag.1 + 1 + tt + z.diff.lag)
```

Residuals:

Min	1Q	Median	3Q	Max
-2.5913	-0.6866	-0.0059	0.6431	3.4217

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.1919181	0.1162438	-1.651	0.09940 .
z.lag.1	-0.0208014	0.0086373	-2.408	0.01640 *
tt	-0.0015026	0.0007122	-2.110	0.03540 *
z.diff.lag	0.1448861	0.0453353	3.196	0.00149 **

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.044 on 477 degrees of freedom

Multiple R-squared: 0.02987, Adjusted R-squared: 0.02377

F-statistic: 4.895 on 3 and 477 DF, p-value: 0.002321

Value of test-statistic is: -2.4083 2.3139 2.9

Critical values for test statistics:

	1pct	5pct	10pct
tau3	-3.98	-3.42	-3.13
phi2	6.15	4.71	4.05
phi3	8.34	6.30	5.36

The ADF test is given by the t-value for the coefficient z.lag.1, which is -2.408. We compare

this to the critical value for tau3. The rejection region is the left tail, so we fail to reject at the all levels of significance since the ADF test = -2.408 is greater than all of the provided tau3 values. We do have a unit root in the series.

This is of course not a proper answer, we need to write down the type of random walk, equations that will be used, define variables in these equations, the null and alternative hypothesis and the form of the test statistic. Remember, R uses the **differenced version** of the ADF, and we will let R determine the value of p , the length of the differenced lag structure. So a thorough answer would be something like:

Since the data has an obvious trend, the random walk model is defined by: $y_t = c_t + \phi_1 y_{t-1} + \epsilon_t$, where $c_t = \omega_0 + \omega_1 t$.

Using the regression: $\Delta y_t = c_t + \phi_{c1} y_{t-1} + \sum_{i=1}^{p-1} \beta_i \Delta y_{t-i} + \epsilon_t$.

We wish to test $H_0 : \phi_{c1} = 0$ against $H_1 : \phi_{c1} < 0$, where $\phi_{c1} = \phi_1 - 1$.

The test-statistic is given by $ADF = \hat{\phi}_{c1} / s.e.(\hat{\phi}_{c1})$.

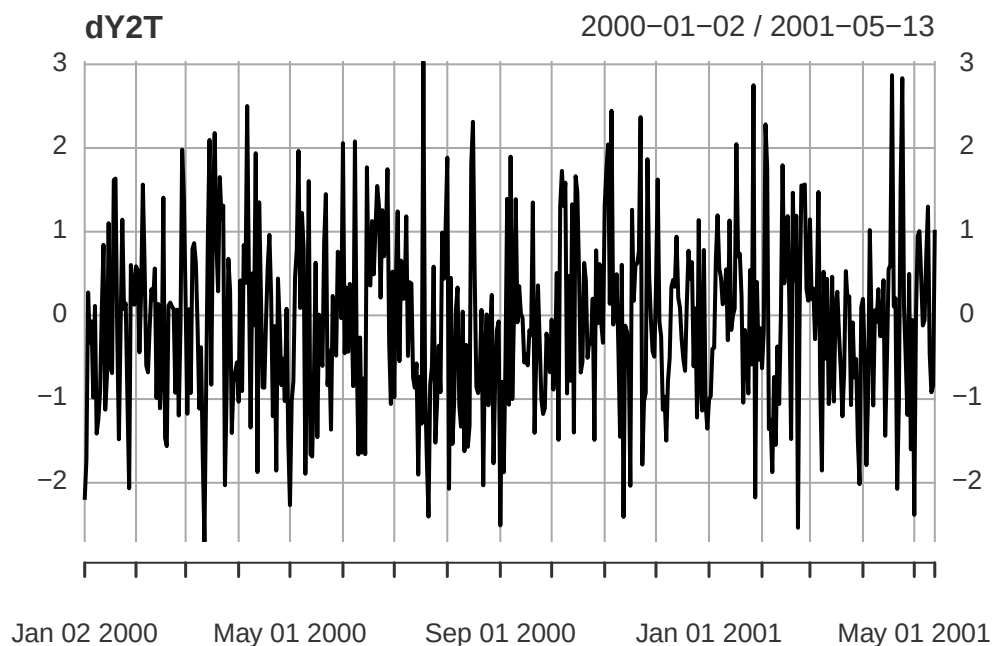
We reject H_0 if the ADF test $<$ tau. At the 5% level of significance, tau = -3.42. Since -2.408 is not less than -3.42, we fail to reject the null. There is insufficient evidence to support the hypothesis that $\phi_{c1} < 0$. We can therefore conclude that the series does have a unit root (*i.e.* the y2t series is not stationary).

NOTE: If $\phi_{c1} = 0$, this means that $\phi_1 - 1 = 0$ which means that $\phi_1 = 1$ or we have a unit root.

3. Create a new variable **dy2t = y2t - y2t-1**. Repeat the Augmented Dickey Fuller test to check whether this new time series contains a unit root.

To remove a unit root (*i.e.* make it stationary), we can difference the variable. So we would expect that the differenced series would not have a unit root, and we would reject the null using the ADF test. We still need to look at the line graph to determine if we include a trend or not.

```
# first difference
dY2T <- na.omit(diff(data.xts$Y2T)) # vector not data.frame
plot(dY2T)
```



Since the data has no obvious trend, the random walk model is defined by: $y_t = c_t + \phi_1 y_{t-1} + \epsilon_t$, where $c_t = \omega_0$. We would select type = "drift" in our ADF test.

```
adf_level_BIC <- ur.df(dY2T, type = "drift", selectlags = "BIC", lags = 17)
summary(adf_level_BIC)
```

```
#####
# Augmented Dickey-Fuller Test Unit Root Test #
#####
```

Test regression drift

Call:

```
lm(formula = z.diff ~ z.lag.1 + 1 + z.diff.lag)
```

Residuals:

Min	1Q	Median	3Q	Max
-2.6583	-0.7140	0.0203	0.6267	3.2851

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.05238	0.04791	-1.093	0.275
z.lag.1	-0.82889	0.06004	-13.805	<2e-16 ***
z.diff.lag	-0.03995	0.04569	-0.875	0.382

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.046 on 477 degrees of freedom

Multiple R-squared: 0.4334, Adjusted R-squared: 0.431

F-statistic: 182.4 on 2 and 477 DF, p-value: < 2.2e-16

Value of test-statistic is: -13.8055 95.2962

Critical values for test statistics:

	1pct	5pct	10pct
tau2	-3.44	-2.87	-2.57
phi1	6.47	4.61	3.79

The ADF test value is -13.805 (the t-value for $z_{\text{lag},1}$), and the value of tau2 is -3.44 at the 1% level. Since $-13.805 < -3.44$, we would reject the null for all levels of significance. There is sufficient evidence to support the hypothesis that $\phi_{c1} < 0$. We can therefore conclude that the differenced series does not contain a unit root (*i.e.* the differenced series is stationary).

This is what we do when we use log-returns. We are taking the log-difference of share price. We know that share price is not stationary. By taking the log difference, we are simply expressing the difference as a percentage.

AR(1) and AR(2) Forecast derivations

AR(1) Forecast Derivation

Suppose the fitted AR(1) model is written in mean form as

$$Y_t - \mu = \phi(Y_{t-1} - \mu) + \varepsilon_t,$$

where

$$\phi = 0.7848, \quad \mu = -0.0426.$$

The last observed value is

$$Y_T = 2.09081693.$$

Hence the fitted model is

$$Y_t - (-0.0426) = 0.7848(Y_{t-1} - (-0.0426)) + \varepsilon_t,$$

or equivalently,

$$Y_t + 0.0426 = 0.7848(Y_{t-1} + 0.0426) + \varepsilon_t.$$

1-step ahead forecast

The 1-step ahead forecast is

$$\hat{Y}_{T+1|T} = \mu + \phi(Y_T - \mu).$$

Substituting the values,

$$\hat{Y}_{T+1|T} = -0.0426 + 0.7848(2.09081693 - (-0.0426))$$

$$= -0.0426 + 0.7848(2.13341693)$$

$$\hat{Y}_{T+1|T} \approx 1.63170561.$$

2-step ahead forecast

For an AR(1) model, replace the unknown future value by its forecast:

$$\hat{Y}_{T+2|T} = \mu + \phi(\hat{Y}_{T+1|T} - \mu).$$

So,

$$\hat{Y}_{T+2|T} = -0.0426 + 0.7848(1.63170561 - (-0.0426))$$

$$= -0.0426 + 0.7848(1.67430561)$$

$$\hat{Y}_{T+2|T} \approx 1.27139504.$$

3-step ahead forecast

Similarly,

$$\hat{Y}_{T+3|T} = \mu + \phi(\hat{Y}_{T+2|T} - \mu).$$

Thus,

$$\hat{Y}_{T+3|T} = -0.0426 + 0.7848(1.27139504 - (-0.0426))$$

$$= -0.0426 + 0.7848(1.31399504)$$

$$\hat{Y}_{T+3|T} \approx 0.98862331.$$

General formula

More generally, for an AR(1) model,

$$\hat{Y}_{T+h|T} = \mu + \phi^h(Y_T - \mu).$$

Hence, in this case,

$$\hat{Y}_{T+h|T} = -0.0426 + (0.7848)^h(2.09081693 - (-0.0426)).$$

Final answers for AR(1)

$$\hat{Y}_{T+1|T} \approx 1.6317,$$

$$\hat{Y}_{T+2|T} \approx 1.2714,$$

$$\hat{Y}_{T+3|T} \approx 0.9886.$$

AR(2) Forecast Derivation

Suppose the fitted AR(2) model is written in mean form as

$$Y_t - \mu = \phi_1(Y_{t-1} - \mu) + \phi_2(Y_{t-2} - \mu) + \varepsilon_t,$$

where

$$\phi_1 = 0.7986, \quad \phi_2 = -0.0175, \quad \mu = -0.0433.$$

The last two observed values are

$$Y_T = 2.09081693, \quad Y_{T-1} = 0.20989475.$$

Hence the fitted model is

$$Y_t - (-0.0433) = 0.7986(Y_{t-1} - (-0.0433)) - 0.0175(Y_{t-2} - (-0.0433)) + \varepsilon_t,$$

or equivalently,

$$Y_t + 0.0433 = 0.7986(Y_{t-1} + 0.0433) - 0.0175(Y_{t-2} + 0.0433) + \varepsilon_t.$$

1-step ahead forecast

The 1-step ahead forecast is

$$\hat{Y}_{T+1|T} = \mu + \phi_1(Y_T - \mu) + \phi_2(Y_{T-1} - \mu).$$

Substituting the values,

$$\hat{Y}_{T+1|T} = -0.0433 + 0.7986(2.09081693 - (-0.0433)) - 0.0175(0.20989475 - (-0.0433))$$

$$= -0.0433 + 0.7986(2.13411693) - 0.0175(0.25319475)$$

$$\hat{Y}_{T+1|T} \approx 1.65657487.$$

2-step ahead forecast

For an AR(2) model, replace the unknown future value by its forecast:

$$\hat{Y}_{T+2|T} = \mu + \phi_1(\hat{Y}_{T+1|T} - \mu) + \phi_2(Y_T - \mu).$$

So,

$$\hat{Y}_{T+2|T} = -0.0433 + 0.7986(1.65657487 - (-0.0433)) - 0.0175(2.09081693 - (-0.0433))$$

$$= -0.0433 + 0.7986(1.69987487) - 0.0175(2.13411693)$$

$$\hat{Y}_{T+2|T} \approx 1.27687303.$$

3-step ahead forecast

Similarly,

$$\hat{Y}_{T+3|T} = \mu + \phi_1(\hat{Y}_{T+2|T} - \mu) + \phi_2(\hat{Y}_{T+1|T} - \mu).$$

Thus,

$$\hat{Y}_{T+3|T} = -0.0433 + 0.7986(1.27687303 - (-0.0433)) - 0.0175(1.65657487 - (-0.0433))$$

$$= -0.0433 + 0.7986(1.32017303) - 0.0175(1.69987487)$$

$$\hat{Y}_{T+3|T} \approx 0.98124237.$$

General recursion

More generally, for an AR(2) model, the forecasts are obtained recursively as

$$\hat{Y}_{T+h|T} = \mu + \phi_1(\hat{Y}_{T+h-1|T} - \mu) + \phi_2(\hat{Y}_{T+h-2|T} - \mu), \quad h \geq 2.$$

Final answers for AR(2)

$$\hat{Y}_{T+1|T} \approx 1.6566,$$

$$\hat{Y}_{T+2|T} \approx 1.2769,$$

$$\hat{Y}_{T+3|T} \approx 0.9812.$$